

# Tutorial 07: Defense

A practical local-server Screeps module

<docs/tutorials/07-defense.md>

## Episode 7: Under Siege

Nothing has attacked this room yet. Your collaborator doesn't treat that as good news – it treats it the way a good CTO treats "we've never had a security incident."

"Every tick this colony runs undefended is a tick you got lucky," it says. "You have a tower slot unlocked and nothing built in it. You have ramparts allowed and none placed. That's not a peaceful room. That's an unpatched one that hasn't been found yet."

## Goal

Give the colony structures that fight back and structures that absorb damage, without adding a single combat creep yet:

- Reach RCL3 and build a tower
- Write the code that makes the tower actually fire
- Place ramparts over key structures and teach the builder to raise their hit points
- Trigger a real test invasion on this local server and watch the tower respond

## Prerequisites

Tutorial 06 is complete:

- `role.miner.js` and `role.hauler.js` are running the colony's energy logistics.
- `role.upgrader.js` and `role.builder.js` are both active.
- The room is at RCL2 or higher.

## Step 1: Reach RCL3

```
Game.spawns.Spawn1.room.controller.level
```

Towers require RCL3. If you're still at RCL2, let the upgrader keep working – this is the same kind of wait as the RCL1-to-RCL2 climb in Episode 4, just a bigger `progressTotal`. Consider spawning a second upgrader temporarily by raising `POPULATION.upgrader` to 2 if the wait feels long; drop it back down once you're past RCL3.

Checkpoint: `Game.spawns.Spawn1.room.controller.level` returns 3.

## Step 2: Build the Tower

```
const room = Game.spawns.Spawn1.room;  
const spawn = Game.spawns.Spawn1;  
room.createConstructionSite(spawn.pos.x + 2, spawn.pos.y, STRUCTURE_TOWER);
```

Adjust the coordinates if that tile is blocked. The builder role from Episode 4 needs no changes to complete this – it was already generic across structure types.

Checkpoint:

```
Game.spawns.Spawn1.room.find(FIND_MY_STRUCTURES, { filter: { structureType: STRUCTURE_TOWER } }).length
```

Expected result: 1, once the builder finishes it.

### Step 3: Make the Tower Actually Fire

A tower does nothing on its own. It has no default behavior – every attack, heal, or repair action has to come from your code, every tick, just like a creep. Add this to `main.js`:

```
function runTowers() {
  const towers = Game.spawns.Spawn1.room.find(FIND_MY_STRUCTURES, {
    filter: { structureType: STRUCTURE_TOWER },
  });

  towers.forEach((tower) => {
    const hostile = tower.pos.findClosestByRange(FIND_HOSTILE_CREEPS);
    if (hostile) {
      tower.attack(hostile);
      return;
    }

    const damaged = tower.pos.findClosestByRange(FIND_MY_STRUCTURES, {
      filter: (structure) => structure.hits < structure.hitsMax,
    });
    if (damaged) {
      tower.repair(damaged);
    }
  });
}
```

Call it from the main loop, after the per-creep dispatch:

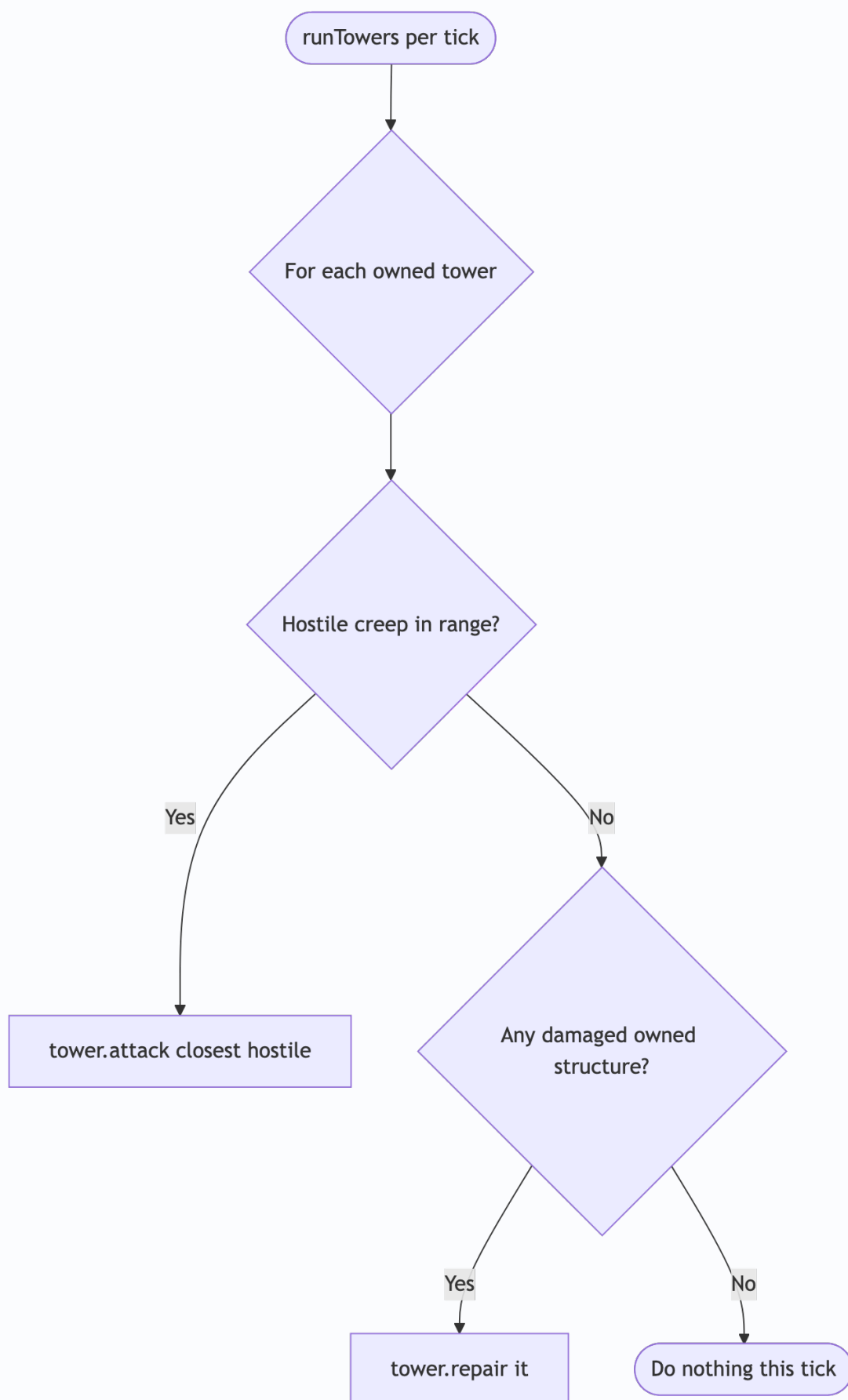
```
module.exports.loop = function () {
  for (const name in Memory.creeps) {
    if (!Game.creeps[name]) {
      delete Memory.creeps[name];
    }
  }

  spawnMissingRoles();

  for (const name in Game.creeps) {
    const creep = Game.creeps[name];
    if (creep.memory.role === 'upgrader') {
      roleUpgrader.run(creep);
    } else if (creep.memory.role === 'builder') {
      roleBuilder.run(creep);
    } else if (creep.memory.role === 'miner') {
      roleMiner.run(creep);
    } else if (creep.memory.role === 'hauler') {
      roleHauler.run(creep);
    }
  }

  runTowers();
};
```

The tower defaults to repairing your own damaged structures when nothing hostile is around, so its energy doesn't sit idle between attacks.



Same shape as every role in this season: check state, act, or wait – a tower just doesn't move.

## Step 4: Keep the Tower Fueled

A tower with no energy can't attack. Give haulers a reason to prioritize it. Update `role.hauler.js`'s delivery target selection:

```
if (creep.memory.hauling) {
  const lowTower = creep.room.find(FIND_STRUCTURES, {
    filter: (structure) =>
      structure.structureType === STRUCTURE_TOWER &&
      structure.store[RESOURCE_ENERGY] < structure.store.getCapacity(RESOURCE_ENERGY) * 0.5,
  })[0];

  const target =
    lowTower ||
    creep.pos.findClosestByPath(FIND_STRUCTURES, {
      filter: (structure) =>
        (structure.structureType === STRUCTURE_EXTENSION || structure.structureType === STRUCTURE_SPAWN)
    &&
      structure.store.getFreeCapacity(RESOURCE_ENERGY) > 0,
    });

  if (!target) {
    return;
  }
  if (creep.transfer(target, RESOURCE_ENERGY) === ERR_NOT_IN_RANGE) {
    creep.moveTo(target, { visualizePathStyle: { stroke: '#ffffff' } });
  }
  return;
}
```

Checkpoint:

```
Game.spawns.Spawn1.room.find(FIND_MY_STRUCTURES, { filter: { structureType: STRUCTURE_TOWER } })
[0].store[RESOURCE_ENERGY]
```

This should climb toward full and stay there, not sit at 0.

## Step 5: Place Ramparts

Ramparts can be built directly on top of your own structures to protect them, or on open tiles to block enemy movement. Protect the spawn and tower:

```
const room = Game.spawns.Spawn1.room;
const spawn = Game.spawns.Spawn1;
const tower = room.find(FIND_MY_STRUCTURES, { filter: { structureType: STRUCTURE_TOWER } })[0];

room.createConstructionSite(spawn.pos.x, spawn.pos.y, STRUCTURE_RAMPART);
room.createConstructionSite(tower.pos.x, tower.pos.y, STRUCTURE_RAMPART);
```

A freshly completed rampart has very low hit points relative to its maximum – it needs to be repaired up before it means anything.

## Step 6: Teach the Builder to Repair

Update `role.builder.js` so that once there's nothing left to build, it repairs instead of standing idle:

```
const roleBuilder = {
  run(creep) {
    if (creep.memory.building && creep.store[RESOURCE_ENERGY] === 0) {
      creep.memory.building = false;
    }
  }
}
```

```

if (!creep.memory.building && creep.store.getFreeCapacity() === 0) {
  creep.memory.building = true;
}

if (creep.memory.building) {
  const site = creep.pos.findClosestByPath(FIND_CONSTRUCTION_SITES);
  if (site) {
    if (creep.build(site) === ERR_NOT_IN_RANGE) {
      creep.moveTo(site, { visualizePathStyle: { stroke: '#ffffff' } });
    }
    return;
  }
}

const damaged = creep.pos.findClosestByPath(FIND_STRUCTURES, {
  filter: (structure) => structure.hits < structure.hitsMax && structure.structureType !==
STRUCTURE_WALL,
});
if (damaged && creep.repair(damaged) === ERR_NOT_IN_RANGE) {
  creep.moveTo(damaged, { visualizePathStyle: { stroke: '#ffffff' } });
}
return;
}

const sources = creep.room.find(FIND_SOURCES);
if (creep.harvest(sources[0]) === ERR_NOT_IN_RANGE) {
  creep.moveTo(sources[0], { visualizePathStyle: { stroke: '#ffaa00' } });
}
},
};
};

module.exports = roleBuilder;

```

`STRUCTURE_WALL` is excluded on purpose. Walls have hit point maximums in the hundreds of millions – a builder that tries to top one off will never do anything else again.

Checkpoint:

```

Game.spawns.Spawn1.room.find(FIND_MY_STRUCTURES, { filter: { structureType:
STRUCTURE_RAMPART } }).map((r) => r.hits)

```

These values should be climbing, not flat.

## Step 7: Trigger a Test Invasion

Waiting for a real, energy-triggered invader to show up is impractical for a tutorial – the automatic trigger fires only after roughly 100,000 energy has been mined in a room, which is a real-time wait measured in hours, not a testing loop. This repo's local server has a faster, fully offline way to force the issue.

`screeps-server/mods/sparring-ground.js` adds a local HTTP endpoint that asks the server's own invasion system to act immediately. From a terminal on the machine running the server:

```

curl -X POST http://localhost:21025/local/api/sparring/wave \
-H 'Content-Type: application/json' \
-d '{"room": "<your-room-name>"}'

```

This doesn't fabricate a creep – it nudges the room's real `invaderGoal` / `invaderHarvested` counters past the threshold and runs the engine's own invasion cronjob, the same mechanism that would eventually fire on its own. See `docs/local-screeps.md` for exactly what it's doing under the hood and its one real limitation (it needs at least one neutral, unclaimed exit to spawn into).

Checkpoint:

- An invader creep appears at one of your room's exits within a few ticks of the call.

- Within a tick or two of it coming into tower range, `runTowers()` calls `tower.attack()` on it – watch the tower's targeting beam in the client.
- The tower's `store[RESOURCE_ENERGY]` visibly drops as it fires.

 **Screenshot placeholder:** A tower's attack beam connecting with a test invader in the client – the single most satisfying image this episode produces, and worth capturing the first time it happens.

```
Game.spawns.Spawn1.room.find(FIND_HOSTILE_CREEPS)
```

Expected result, while the invader is alive: an array with one creep in it, not empty.

If the endpoint doesn't produce an invader – a future engine update changed something the mod depends on, for instance – the guaranteed fallback is the client's own **Invasion** tab in the room side panel: click **Create an invader**, then click an exit tile (it highlights differently once you're hovering a valid one). That's a first-party client feature, not something this repo has to maintain.

If this server has an NPC bot placed nearby ( `docs/local-screeps.md` covers `bots.spawn(...)` ), you may see `runTowers()` fire on its creeps too, whenever they wander close enough – that's real, unscripted hostile contact on top of whatever you trigger on demand.

## Troubleshooting

If `createConstructionSite` fails for the tower, confirm RCL3 and check the tile isn't already occupied.

If the tower never fires in a room where you can confirm a hostile is present, check `tower.store[RESOURCE_ENERGY]` – an empty tower silently does nothing on `.attack()`.

If ramparts block your own creeps' movement, they shouldn't – ramparts only block creeps belonging to other players by default. If your own creeps can't pass, check whether the rampart's `public` flag or ownership is set correctly.

If `curl http://localhost:21025/local/api/sparring/health` doesn't return `{"ok":1,...}`, the mod didn't load – confirm `screeps-server/mods/sparring-ground.js` exists and restart the container ( `docker restart autonate-screeps` ).

If the wave endpoint returns `ok: 1` but no invader appears, the room's exits may all lead to claimed or reserved rooms – check with `Game.map.describeExits(room)` and the intel from Episode 11 if you have it yet, or fall back to the Invasion panel.

## Completion Criteria

You are done when:

- The room is RCL3 or higher with one completed tower.
- `runTowers()` is running every tick as part of `main`.
- The hauler prioritizes a low-energy tower over extensions.
- At least one rampart is placed and its hit points are increasing under the builder's repair logic.
- You've triggered a test invasion through the sparring-ground endpoint (or the Invasion panel) and watched the tower actually fire on it.

## Learning Notes

After completing the tutorial, write down:

- What's the difference between a structure that "just works" (like a container) and one that requires code every tick to do anything (like a tower)?
- If two hostiles entered at once, would `findClosestByRange` pick the more dangerous one or just the nearer one? Is that the right default?
- Why exclude walls from the builder's repair logic instead of just giving walls a repair priority of last?
- What would you need to add to this room's defenses before you'd feel comfortable expanding to a second room?

- How many ticks passed between the test invader spawning and the tower's first shot? Was the tower already fueled, or did the hauler have to catch up?

## Next: Episode 8 – Beyond the Border

The colony can survive an attack now – you watched it happen. It still only exists in one room.

"You've been treating this room like the whole world," your collaborator says. "It's one room. There are exits on all four sides, and every one of them leads somewhere you haven't looked yet – the 'have you considered a second market' conversation, except the market is an actual adjacent room."

See `docs/roadmap.md` for the full season.